

kube-prometheus-stack Scaling Guide and values-scaled.yaml

Scaling kube-prometheus-stack for Heavy Query Load

This document explains how to scale the kube-prometheus-stack when Prometheus queries regularly exceed 30 seconds and you need higher throughput, more memory, and longer timeouts.

1. Why Queries Take >30 Seconds

Common causes:

Cause	What to do
Default query timeout	Prometheus default is 2m; Grafana datasource often uses 30s. Increase both so long-running queries can complete.
Single replica	All read load hits one pod. Add replicas (and optionally shards) to spread load.
Insufficient CPU/memory	Prometheus is CPU- and memory-heavy for ingestion and queries. Set requests/limits.
Large retention / no retention size cap	More data = slower queries and more disk. Tune retention and set retention size.
No persistent storage or small disk	WAL and TSDB need fast, sufficient storage. Use PVC with adequate size and class.
Too many targets/series	High cardinality increases memory and query cost. Use metric relabeling, drop unused metrics, or sharding.
Expensive recording/alert rules	Heavy rules increase load. Optimize or split evaluation.

2. Prometheus Server Scaling

2.1 Query timeouts and limits (critical for >30s queries)

Configure **QuerySpec** under `prometheus.prometheusSpec.query` so Prometheus allows longer and heavier queries:

```
prometheus:
  prometheusSpec:
    # Allow queries to run up to 5 minutes (300s) instead of
    # default 2m
    query:
      timeout: 5m
      # Optional: increase max samples per query (default 500000000)
      # query:
      #   maxSamples: 1000000000
      # Optional: lookback delta for instant queries (default 5m)
      # query:
      #   lookbackDelta: 10m
```

- **timeout:** Maximum evaluation time per query. Set to 5m or higher if you need queries >30s.
- **maxSamples:** Raise if you hit “too many samples” errors on large range queries.
- **lookbackDelta:** Affects instant-query lookback; increase only if needed.

2.2 Replicas and shards

- **Replicas:** Same data on each replica; good for read scaling and HA. Queries are spread across replicas by the service.

```
prometheus:
  prometheusSpec:
    replicas: 2
```

- **Shards:** Split targets across N Prometheus instances (each has different data). Use when a single instance has too many targets/series. Total pods = `replicas × shards`. For global view you need Thanos (or similar) or remote read.

```
prometheus:
  prometheusSpec:
    replicas: 1
    shards: 2
```

Start with **replicas: 2** for read scaling; add **shards** only if one instance is overloaded by scrape load.

2.3 Resources

Set requests and limits so Prometheus isn't throttled or OOMKilled. Scale with retention and number of series.

```
prometheus:
  prometheusSpec:
    resources:
      requests:
        cpu: 500m
        memory: 2Gi
      limits:
        cpu: 2000m
        memory: 8Gi
```

Rough guidance: 1–2 GB RAM per 100k active series; increase for long retention and heavy queries.

2.4 Retention and storage

- **retention:** Lower retention = less data = faster queries and less disk. Balance with how far back you need to query.
- **retentionSize:** Prevents unbounded disk growth.

```
prometheus:
  prometheusSpec:
    retention: 15d
    retentionSize: "45GB"
```

Use **storageSpec** so data survives restarts and has enough IOPS:

```
prometheus:
  prometheusSpec:
    storageSpec:
      volumeClaimTemplate:
        spec:
          storageClassName: fast-ssd # or your preferred class
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 50Gi
```

2.5 Scrape and evaluation intervals

- **scrapeInterval:** Default 30s. Increasing (e.g. 60s) reduces scrape load and storage; decreasing increases resolution and load.
- **evaluationInterval:** How often alert/recording rules run. Increasing reduces CPU.

```
prometheus:
  prometheusSpec:
    scrapeInterval: 30s
    scrapeTimeout: 10s
    evaluationInterval: 30s
```

2.6 Query log (optional)

Useful to see which queries are slow:

```
prometheus:
  prometheusSpec:
    queryLogFile: /var/log/prometheus/query.log
```

You must add a volume/writeable path for this (e.g. emptyDir or PVC).

2.7 Pod placement

Spread Prometheus pods across nodes/zones for HA:

```
prometheus:
  prometheusSpec:
    podAntiAffinity: hard
    podAntiAffinityTopologyKey: kubernetes.io/hostname
```

For multi-zone:

```
prometheus:
  prometheusSpec:
    podAntiAffinityTopologyKey: topology.kubernetes.io/zone
```

3. Grafana datasource timeout

Grafana's Prometheus datasource has its own timeout. If it's 30s, Grafana will give up before Prometheus. Set it to at least the Prometheus query timeout (e.g. 300 seconds for 5m):

```
grafana:
  sidecar:
    datasources:
      timeout: 300
```

timeout is in seconds. Use the same (or slightly higher) value as `prometheus.prometheusSpec.query.timeout` (e.g. 300 for 5m).

4. Prometheus Operator

The operator reconciles Prometheus CRs. Under heavy or many Prometheus instances, give it more resources:

```
prometheusOperator:
  resources:
    requests:
      cpu: 100m
      memory: 150Mi
    limits:
      cpu: 500m
      memory: 500Mi
```

5. Alertmanager (optional)

For HA and more capacity:

```
alertmanager:
  alertmanagerSpec:
    replicas: 2
    resources:
      requests:
        cpu: 50m
        memory: 64Mi
      limits:
        cpu: 200m
        memory: 128Mi
```

6. Recording rules and cardinality

- **Recording rules:** Pre-aggregate expensive queries so dashboards use the recorded metric instead of the heavy expression. This reduces query load and latency.
- **Metric relabeling:** Drop high-cardinality or unused metrics to reduce series count and memory.

Reducing cardinality and moving heavy logic into recording rules often has the biggest impact on query time.

7. Thanos / remote write (optional)

For very large or long-retention setups:

- **Thanos sidecar:** Offloads blocks to object storage and allows a Thanos Query in front of Prometheus. Use when you need long retention and a single query endpoint across shards/replicas.
- **Remote write:** Send samples to a dedicated backend (e.g. Cortex, Mimir, Thanos Receive) and keep Prometheus retention short. Queries then go to the backend.

These are larger architectural changes; start with the tuning above before adding Thanos or remote write.

8. Checklist summary

☐

Set `prometheus.prometheusSpec.query.timeout` (e.g. 5m) so queries can run >30s.

☐

Set `grafana.sidecar.datasources.timeout` (e.g. 300) to match.

☐

Set `prometheus.prometheusSpec.resources` (CPU and memory).

☐

Set `prometheus.prometheusSpec.retention` and `retentionSize`.

☐

Configure `prometheus.prometheusSpec.storageSpec` with adequate PVC size and class.

☐

Consider `prometheus.prometheusSpec.replicas`: 2 (and optionally shards).

☐

Optionally set `prometheus.prometheusSpec.podAntiAffinity` for HA.

☐

Tune scrape/evaluation intervals and optimize recording rules/cardinality as needed.

9. Applying the sample values

The **values-scaled.yaml** overlay below contains a minimal overlay focused on scaling and longer query timeouts. Use it on top of the default values:

```
helm upgrade --install kube-prometheus-stack . \
  -f values.yaml \
  -f values-scaled.yaml \
  -n monitoring
```

Adjust `values-scaled.yaml` for your cluster size, storage class, and retention needs. After upgrading, monitor Prometheus and Grafana for CPU, memory, and query latency.

Appendix: values-scaled.yaml (full file)

```
# values-scaled.yaml – overlay for scaling kube-prometheus-stack
# Use with: helm upgrade -f values.yaml -f values-scaled.yaml ...
# Addresses: queries >30s, higher load, and better availability.

# --- Prometheus Server ---
prometheus:
  prometheusSpec:
    # Allow long-running queries (e.g. 5m); default is 2m
    query:
      timeout: 5m
      # Uncomment if you hit "max samples" limits on heavy range
queries:
      # maxSamples: 1000000000

    # Read scaling: 2 replicas share query load (same data on
each)
    replicas: 2

    # Retention and size cap
    retention: 15d
    retentionSize: "45GB"

    # Scrape/evaluation (adjust if you need different resolution)
    scrapeInterval: 30s
    scrapeTimeout: 10s
    evaluationInterval: 30s

    # Resources – scale up with series count and retention
    resources:
      requests:
        cpu: 500m
        memory: 2Gi
      limits:
```

```

        cpu: 2000m
        memory: 8Gi

# Persistent storage – set storageClassName to your fast disk
class
storageSpec:
  volumeClaimTemplate:
    spec:
      storageClassName: "" # e.g. fast-ssd, gp3, standard
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 50Gi

# Spread replicas across nodes
podAntiAffinity: soft
podAntiAffinityTopologyKey: kubernetes.io/hostname

# Optional: enable query logging to debug slow queries
(ensure volume exists)
# queryLogFile: /var/log/prometheus/query.log

# --- Grafana datasource timeout (must be >= Prometheus query
timeout) ---
# Timeout in seconds; 300 = 5 minutes to match query.timeout: 5m
grafana:
  sidecar:
    datasources:
      timeout: 300

# --- Prometheus Operator (optional, for larger clusters) ---
prometheusOperator:
  resources:
    requests:
      cpu: 100m
      memory: 150Mi
    limits:
      cpu: 500m
      memory: 500Mi

# --- Alertmanager HA (optional) ---
alertmanager:
  alertmanagerSpec:
    replicas: 2
    resources:
      requests:
        cpu: 50m
        memory: 64Mi

```



```
limits:  
  cpu: 200m  
  memory: 128Mi
```